

Phishing Website Detection using Multi-Objective Feature Selection

Finding the fewest, fastest checks needed to catch a scam website

Algorithm: SPEA2 Dataset: Vrbančič Phishing Dataset Status: Complete

OBJECTIVES

1. To minimise the time required to detect phishing website

2. To maximize recall value

A third quantity, the false-alarm rate, is optimised alongside these two. It is not a goal in its own right but is required to make objective 2 well-posed — see Section 4.

1. Introduction

A phishing website is a fake site built to look like a real one — a bank, a college portal, a payment page — so that a visitor types in their password or card details and hands them to an attacker.

Software that inspects a link and judges whether it is genuine faces a constraint ordinary classification problems do not: **it must decide before the user acts**. A detector taking two seconds is useless, because the password has already been typed. So it must be both **accurate** and **fast**, and these pull against each other, since every extra check adds a little accuracy and a little delay. This project studies that trade-off directly: we build a phishing detector, then use a soft-computing optimisation algorithm to find which of its 111 checks are actually worth performing.

2. Dataset

Vrbančič Phishing Dataset — G. Vrbančič, I. Fister Jr., V. Podgorelec, published in *Data in Brief*, Vol. 33, 2020 (a peer-reviewed data journal). DOI 10.1016/j.dib.2020.106438. Available free as CSV from github.com/GregaVrbancic/Phishing-Dataset, no account required.

TABLE 1 — DATASET VARIANTS

File	Rows	Legitimate (0)	Phishing (1)	Features	Use
dataset_small.csv	58,645	27,998	30,647	111	Primary — balanced classes
dataset_full.csv	88,647	58,000	30,647	111	Optional extra test

112 columns total = 111 feature columns + 1 answer column named `phishing`, where 0 = legitimate and 1 = phishing. We use the balanced file so that our error measurements are directly comparable and the algorithm cannot score well simply by favouring the larger class.

2.1 What the 111 features measure

Crucially, **the dataset never opens the webpage**. Every feature comes from the web address itself or from public records about the domain. The features split into two kinds with vastly different costs.

TABLE 2 — FEATURE GROUPS AND THEIR COST

Kind	Count	What it measures	Cost
Read from the URL text whole URL (19), domain (21), directory (18), file (18), parameters (20)	96	Counts of dots, hyphens, @, slashes etc. within each part of the address, plus lengths. The same counts are repeated for five sections	≈ 0.01 ms total
External lookups	15	Domain age and expiry (WHOIS), DNS records, SSL certificate, Google indexing	≈ 1,320 ms each needs a network query
Total	111		

WHY THIS SPLIT IS THE HEART OF THE PROJECT

The 15 lookup features cost **over a hundred thousand times** more than the 96 text features. So a detector's speed is decided almost entirely by *how many network lookups it needs*, not by how many features it uses. If our optimisation can discard the lookup features, the detector becomes fast enough to run inside a web browser — which is exactly what Objective 1 asks for.

2.2 Why this dataset was chosen

It is large enough (58,645 sites), peer-reviewed and citable, free, and needs no page download — so detection can happen before the page loads. Most importantly:

KEY JUSTIFICATION — EXPECT A QUESTION HERE

An older 30-feature phishing dataset (UCI) is the field's traditional benchmark, and we deliberately did **not** use it as our primary data. With 30 features there are about 1 billion possible combinations, which simpler search methods handle adequately — leaving us unable to justify an advanced optimisation algorithm at all.

With **111 features there are $2^{111} \approx 2.6 \times 10^{33}$** combinations. No computer can test them all, and no simple method searches that space well. **This is what makes an intelligent search algorithm genuinely necessary rather than decorative** — which is the point of a soft computing project.

3. Aim

THE RESEARCH QUESTION

Out of 111 possible checks, what is the smallest and fastest set that still reliably catches phishing websites — and what exactly does speed cost us in accuracy?

The classifier is the instrument, not the subject. We are not inventing a better classification model; we use a standard one (Random Forest) and hold it fixed. What we study is *which inputs it needs*. Nor are we looking for one answer: because speed and accuracy conflict, there is no single best configuration. We produce the **full set of best possible trade-offs**, so a deployment can pick the point that suits it.

4. The Objectives in Measurable Form

The two objectives stated on page 1 are expressed below as quantities the algorithm can optimise, together with the third that supports them. All are written so **lower is better**, which keeps the mathematics consistent.

TABLE 3 — THE OBJECTIVES AS OPTIMISED

Objective	Definition	Why it matters	Target
f_1 — Detection time Objective 1	Sum of the <i>distinct</i> network lookups the selected features require	Detection must finish before the user enters any information. Range ≈ 0.01 ms (URL text only) to $\approx 1,320$ ms (all features)	As low as possible
f_2 — Missed phishing Objective 2	$FNR = FN / (FN + TP)$ $= 1 - \text{recall}$	Minimising this is the same as maximising recall. The most damaging error — a missed phishing site can cost a user their password, money or identity	$FNR \leq 5\%$ (email-phishing studies: 4–6%)
f_3 — False alarms supporting	$FPR = FP / (FP + TN)$	Blocking real websites frustrates users, who then disable the tool — leaving them less protected than before	$FPR \leq 2\%$ (best email-phishing result: 0.1%)

f_2 and f_3 both come from the same confusion matrix, so measuring them costs nothing extra.

TWO QUESTIONS TO BE READY FOR

Why is f_3 needed if it is not one of our stated objectives? Without it the optimiser finds a trivial cheat: label *every* website as phishing. Recall then reaches 100%, Objective 2 scores perfectly, and the detector is useless. f_3 closes that loophole — it is what makes maximising recall a meaningful goal rather than a degenerate one.

Why not simply count features for f_1 ? Counting would rank a 40-feature URL-only subset (0.01 ms) as worse than a 5-feature subset needing a WHOIS query (200 ms) — when the first is about 20,000 times faster. Because features *share* lookups, cost must be measured by the lookups required, not the feature count.

4.1 Why the objectives conflict

Fewer features means faster detection but less information to judge on. Catching more phishing means flagging more sites, which blocks more real ones. **No single configuration is best on all three** — improving one always costs something on another. The correct output is therefore not one answer but a set of best-possible compromises.

5. Algorithm — SPEA2

5.1 Why an ordinary optimisation algorithm will not do

The obvious approach combines the objectives into one score using weights — $\text{score} = w_1 \cdot (\text{time}) + w_2 \cdot (\text{missed}) + w_3 \cdot (\text{false alarms})$. This is called *scalarisation*, and we rejected it for three reasons:

1. **The weights would be arbitrary.** There is no principled way to decide speed is worth 0.6 and recall 0.3; the answer would be driven by our guess.
2. **It returns only one answer** — one point on the trade-off curve. Changing priorities means running everything again.
3. **It cannot reach some solutions at all.** A weighted sum only finds solutions on outward-curving parts of the trade-off surface; those in an inward-curving region are unreachable *for any weights whatsoever*. This is a mathematical property of the method, not something better tuning can fix.

5.2 What SPEA2 is and how it works

SPEA2 — Strength Pareto Evolutionary Algorithm 2 (Zitzler, Laumanns and Thiele, 2001) is a multi-objective evolutionary algorithm. *Evolutionary* means it works like natural selection: it holds a population of candidate answers, keeps the better ones, combines and mutates them into a new generation, and repeats. *Multi-objective* means it optimises several goals at once without merging them into a single score.

Each candidate is a list of 111 yes/no decisions — for each feature, keep it or drop it. To judge a candidate we train the classifier using only its selected features and measure the objectives.

- ❶ **Dominance.** Candidate A *dominates* B if A is at least as good on all three objectives and strictly better on at least one. Dominated candidates are discarded.
- ❷ **Strength.** Each candidate is given a strength value — the number of others it dominates.
- ❸ **Raw fitness.** A candidate's score is the sum of the strengths of everything that dominates it. Zero means nothing dominates it — it is one of the best found.
- ❹ **Density.** Many candidates score zero, so ties break by how crowded each one's neighbourhood is. Sparse regions are preferred, spreading results evenly instead of clustering.
- ❺ **Archive.** A fixed-size store holds the best candidates found so far. When it overflows, the most crowded entries are removed, keeping the spread wide.

The output is a *Pareto front* — the set of candidates that cannot be improved on one objective without being made worse on another. This set of best possible trade-offs is the project's main result.

5.3 Why SPEA2 rather than the alternatives

1. **It handles duplicate solutions consistently, which our data produces constantly.** Our dataset counts the same characters across five sections of the URL, so features are heavily repetitive and many *different* selections give *identical* results. SPEA2 gives identical candidates identical scores; NSGA-II can give them different scores. Published research identifies duplicate solutions as a specific cause of poor NSGA-II performance in feature selection.
2. **Its usual disadvantage does not apply to us.** SPEA2 is computationally heavier than NSGA-II, but here each candidate requires training a classifier (a few hundred milliseconds) while the algorithm's own bookkeeping takes microseconds — so the difference is invisible.
3. **Recent research favours it.** A 2025 IJCAI paper, "*Proven Approximation Guarantees in Multi-Objective Optimization: SPEA2 Beats NSGA-II*", demonstrates stronger theoretical guarantees for SPEA2.

Rejected alternatives: single-objective PSO and GA handle one objective only; NSGA-III is built for four or more; MOEA/D uses weight vectors, close to the scalarisation we rejected. **NSGA-II we still run**, as our comparison method, since it is the field's standard reference point.

6. Methodology

- ❶ **Load and audit.** Verify $58,645 \times 112$. Investigate the -1 values in the lookup columns — these mark *failed lookups*, not real measurements.
- ❷ **Split the data.** 80% training, 20% testing, stratified. The test set is then sealed.

- ③ **Build the cost model.** Assign every feature to its lookup type; implement the cost calculation.
- ④ **Establish the baseline.** Train on all 111 features, record all objectives. Every later result is compared against this.
- ⑤ **Run SPEA2.** Population 100, 50 generations, repeated with 5 random seeds.
- ⑥ **Run comparisons.** NSGA-II and a weighted-sum PSO under identical conditions, to show what SPEA2 gained us.
- ⑦ **Analyse the front.** Which features survive? How many lookup features were dropped?
- ⑧ **Final validation.** Evaluate the chosen solutions once on the sealed test set.

6.1 How one candidate is scored

```
if no features selected → return worst possible on all objectives

f1 = detection cost      (from the selection; no training needed)
    train classifier on the selected features
f2 = FN / (FN + TP)      (from the confusion matrix)
f3 = FP / (FP + TN)      (from the same confusion matrix)
```

Objective f_1 needs no model training at all, and f_2 and f_3 both come from a single confusion matrix. **So evaluating three objectives costs exactly the same as evaluating one** — optimising for all of them is computationally free here.

6.2 Preventing data leakage

THE KEY METHODOLOGICAL SAFEGUARD

Feature selection is a decision learned from data, so it must never see the test set. If we selected features using all the data and only afterwards split it, our results would be invalid. **Our order is: split first, select features using only the training data, and evaluate on the test set exactly once at the end.**

6.3 How results are measured, and tools

Because multi-objective algorithms return a *set* of answers, comparing them by accuracy is meaningless. The standard measure is **hypervolume** — how much of the objective space a Pareto front covers; larger is better. We also report all objectives for each selected solution, with recall, precision and the confusion matrix.

Tools: Python 3 · pymoo (SPEA2, NSGA-II, hypervolume) · scikit-learn (Random Forest, metrics, splitting) · pandas, numpy, matplotlib. All of it runs on an ordinary laptop or in Google Colab. **No hardware, no web crawling and no paid services are required.**

Dataset. G. Vrbančič, I. Fister Jr., V. Podgorelec. *Datasets for Phishing Websites Detection*. Data in Brief, Vol. 33, 2020. DOI: 10.1016/j.dib.2020.106438

Algorithm. E. Zitzler, M. Laumanns, L. Thiele. *SPEA2: Improving the Strength Pareto Evolutionary Algorithm*. TIK Report 103, ETH Zurich, 2001.

7. Comparison of Algorithms

Each method's feature subset retrains the same Random Forest as the baseline. Every row is measured **once, on the sealed 20% test set (11,729 websites)**.

TABLE 4 — COMPARISON OF METHODS ON THE SEALED TEST SET

Method	Features	Cost	Accuracy	Recall	FPR	Notes
All features (baseline)	111	1,320 ms	95.71%	96.39%	5.04%	Reference point
SPEA2 (gateway)	37	490 ms	95.85%	96.52%	4.89%	Primary; full Pareto front
SPEA2 (no lookups)	48	0.01 ms	90.51%	89.75%	8.66%	Browser-deployable
NSGA-II (gateway)	46	290 ms	95.61%	96.26%	5.11%	Comparison; full Pareto front
NSGA-II (no lookups)	53	0.01 ms	90.73%	90.23%	8.71%	Browser-deployable
PSO weighted (recall)	60	260 ms	95.37%	95.86%	5.16%	1 point/run; weights guessed
PSO weighted (cost)	50	0.01 ms	90.59%	89.98%	8.75%	Separate run needed
PCA (95% var.)	40 comp.	1,320 ms	94.47%	95.25%	6.39%	Blended; needs all 111 inputs
MI filter (top 25)	25	0.01 ms	87.93%	85.90%	9.86%	Ignores interactions

TABLE 5 — SEARCH QUALITY ACROSS 5 INDEPENDENT RUNS

Algorithm	Hypervolume (mean $\pm$ std)	Pareto solutions	Network-free solutions	Output per run
SPEA2	1.155 $\pm$ 0.067	201	27	Whole trade-off curve
NSGA-II	1.186 $\pm$ 0.008	241	32	Whole trade-off curve
PSO weighted	—	1	—	A single point

8. Conclusion

SPEA2 reduced the detector from **111 features to 37** and its detection cost from **1,320 ms to 490 ms (–63%)**, while matching the full-feature baseline on unseen data — recall 96.52% against 96.39%, and fewer false alarms (4.89% against 5.04%). Both stated objectives are therefore met at this point: detection is substantially faster, and recall is not reduced.

- **Instant detection has a measurable price.** A solution using only the URL text (0.01 ms, no network lookups) exists, but recall falls to about 90%. DNS lookups were the most valuable network check; TLS, redirect and Google-index lookups were almost never selected.
- **Multi-objective search was the right approach.** SPEA2 and NSGA-II returned the whole trade-off curve per run; weighted-sum PSO returned one point. The filter method did worst (85.90% recall).
- **Algorithm comparison, reported honestly.** NSGA-II achieved a higher and far more consistent hypervolume than SPEA2, so our prediction that SPEA2 would search better was not supported. At the deployable points, however, the two differ by under 0.3 percentage points.
- **The result is robust.** When all network latencies were scaled by $\div 3$ and $\times 3$, the best recall found stayed between 95.6% and 95.8%.

Limitations: the false-alarm rate (4.89%) stays above the 2% target; results come from one dataset with subsets tuned for a Random Forest; and failed DNS lookups showed an unexpected link to legitimate sites worth checking on a second dataset.